
Captura de movimientos para la detección de
gestos en entornos virtuales
Motion capture for gesture detection in virtual
environments



Trabajo de Fin de Grado
Curso 2025–2026

Autor

Alejandro Barrachina Argudo, Pablo Sánchez Martín

Director

Alejandro Romero Hernández, Ismael Sagredo Olivenza

Colaborador

Grado en Ingeniería Informática y Grado en Desarrollo
de Videojuegos

Facultad de Informática

Universidad Complutense de Madrid

Captura de movimientos para la detección
de gestos en entornos virtuales
Motion capture for gesture detection in
virtual environments

Trabajo de Fin de Grado en Ingeniería Informática y Grado
en Desarrollo de Videojuegos

Autor

Alejandro Barrachina Argudo, Pablo Sánchez Martín

Director

Alejandro Romero Hernández, Ismael Sagredo Olivenza

Colaborador

Convocatoria: *Junio 2026*

Calificación Alejandro Barrachina Argudo: *9.8*

Calificación Pablo Sánchez Martín: *9.8*

Grado en Ingeniería Informática y Grado en Desarrollo de
Videojuegos

Facultad de Informática

Universidad Complutense de Madrid

30 de julio de 2026

Dedicatoria

*A toda la gente que nos ha hecho llegar hasta
aquí,
a los que nos han apoyado y a los que nos han
hecho crecer.
A todos los que han estado a nuestro lado.*

Agradecimientos

A mis padres y mi pareja, por estar siempre a mi lado y apoyarme en todo momento. A mi gato Umbreon, por ser mi compañero de programación, y por haber escrito casi más líneas de código que yo sentándose en el teclado. A mis juntas anteriores de LAG, por acompañarme estos años. A la gente que conocí por LAG, por haberme apoyado en los proyectos de la asociación. A Poletti, Pascal y Blanca, por haberme dado la oportunidad de hacer charlas en la facultad, de desarrollar mis conocimientos más allá de las clases y poder ayudar a más personas a buscar más conocimiento. A la gente de cafetería, por cuidarnos tan bien a todos. Y a todos los voluntarios que vinieron a hacer las pruebas para generar datos, sin ellos este trabajo no hubiera obtenido los resultados que se han conseguido.

Alejandro Barrachina Argudo

A mi madre, mi hermana, mi pareja, mi cuñado, mis tíos y mis primos por haberme apoyado y ayudado a seguir adelante. A mi grupo del instituto por estar siempre ahí para lo que necesitase. A la gente que he conocido en la universidad: tanto LAG como mi clase, por hacer que mis días en la facultad hayan sido más llevaderos. A la gente de cafetería, por cuidarme tanto en estos años. Al despacho 409 (y allegados) por soportarme todos los días, ser mis compañeros de cafetería y ayudarme cuando lo he necesitado.

Pablo Sánchez Martín

Resumen

Captura de movimientos para la detección de gestos en entornos virtuales

Este estudio se centra en la comparativa de distintos modelos de [IA](#) para la detección de gestos específicos mediante el traje de captura de movimiento Perception Neuron 3 con el fin de crear interacciones con [NPCs](#) a través de la comunicación no verbal y la creación de herramientas derivadas de los modelos que ayuden a la comunicación no verbal en entornos virtuales.

Para ello primero se buscaron datasets públicos de gestos, no encontrándose ninguno que se adaptara a las necesidades del estudio. Por ello se creó una herramienta para traducir animaciones de Mixamo a [CSVs](#) para que pudieran ser entendidas por los modelos a entrenar. Esta herramienta generó un gran número de animaciones pero muy desbalanceadas en el número de animaciones de cada tipo.

Para solucionar el problema del desbalance, se realizó una serie de pruebas con usuarios (N=65) en las cuales se les pidió que realizaran 3 tomas de cada gesto mientras llevaban el traje de captura de movimiento puesto. Esta recolección de datos resultó en un total de 975 animaciones, habiendo un total de 195 gestos humanos de correr, saludar, señalar, pegar y sentarse. Se omitió el gesto de baile en las pruebas por el desbalanceo en el dataset generado por ordenador. Esta omisión sirvió para equilibrar más el número de animaciones, pero no para terminar del todo con el desbalanceo en los datos estandarizados.

Una vez se tuvo el dataset, se implementaron distintos modelos de [IA](#) para la detección de gestos bajo una interfaz web para su facilidad de uso. Estos modelos fueron: [LSTM](#), [CNN](#), [RNN](#) y [Random Forest](#). Tras realizar los entrenamientos de todos los modelos, se observó que el modelo [Random Forest](#) era con diferencia el que mejores datos obtenía, con un 95 % de precisión en entrenamiento y un 86 % en test. Los resultados de los modelos basados en redes neuronales fueron bastante menores, pudiendo ser esto así por la falta de datos, de tiempo de entrenamiento y de hardware suficiente para explotar mejor los modelos.

Finalmente, se exportó el modelo [Random Forest](#) a TensorFlow Serving para posteriormente ser utilizado en una aplicación de demostración con un [NPC](#) reactivo. Esta aplicación permite al usuario ver como el [NPC](#) interpreta distintos gestos y responde de manera simple ante ellos.

Palabras clave

Captura de movimiento, Unity, Inteligencia Artificial, Comunicación no verbal, Perception Neuron, TensorFlow, YDF, Keras, Realidad Virtual

Abstract

Motion capture for gesture detection in virtual environments

This study focuses on comparing different [AI](#) models for detecting specific gestures using the Perception Neuron 3 motion capture suit, aiming to create interactions with [NPCs](#) through non-verbal communication and to develop tools derived from the models that enhance non-verbal communication in virtual environments.

To achieve this, public gesture datasets were first sought, but none were found that met the study's needs. Therefore, a tool was created to convert Mixamo animations into [CSVs](#) format so they could be understood by the models to be trained. This tool generated a large number of animations, but they were highly imbalanced in the number of animations for each type.

To address the imbalance issue, a series of user trials (N=65) were conducted, where participants were asked to perform three instances of each gesture while wearing the motion capture suit. This data collection resulted in a total of 975 animations, with 195 human gestures for running, greeting, pointing, punching, and sitting. The gesture of dancing was omitted from the trials due to the imbalance in the dataset generated by computer animations. This omission helped balance the number of animations but did not completely eliminate the imbalance in the standardized data.

Once the dataset was established, various [AI](#) models for gesture detection were implemented under a web interface for ease of use. These models included: [LSTM](#), [CNN](#), [RNN](#), and [Random Forest](#). After training all the models, it was observed that the [Random Forest](#) model significantly outperformed the others, achieving 95% accuracy in training and 86% in testing. The results from the neural network-based models were considerably lower, likely due to insufficient data, limited training time, and hardware constraints that hindered better model performance.

Finally, the [Random Forest](#) model was exported to TensorFlow Serving to be used in a demonstration application featuring a reactive [NPC](#). This application allows users to see how the [NPC](#) interprets different gestures and responds simply to them.

Keywords

Motion capture, Unity, Artificial Intelligence, Non-verbal communication, Perception Neuron, TensorFlow, YDF, Keras, Virtual Reality

Índice

1. Introducción	1
1.1. Motivación	1
1.2. Objetivos	2
1.3. Plan de trabajo	2
2. Estado de la Cuestión	5
2.1. Entornos virtuales	5
2.2. Comunicación no verbal en entornos virtuales	6
2.3. Captura de movimiento	6
2.3.1. Captura de movimiento óptica	7
2.3.2. Captura de movimiento mecánica	7
2.3.3. Captura de movimiento magnética	8
2.4. Motores de videojuegos	8
2.4.1. Unity	8
2.4.2. Unreal	9
2.5. Modelos de IA	10
2.5.1. Tecnologías de IA	10
2.5.2. Librerías de IA	10
2.5.2.1. Tensorflow	10
2.5.2.2. PyTorch	10
2.5.2.3. Scikit-learn	11
2.5.2.4. YDF	11
2.5.3. Modelos de reconocimiento de gestos y poses	11
2.5.3.1. YoLoV11	11
2.5.3.2. Modelos para la detección del balance de personas andando	12
2.5.3.3. Otros modelos de detección de gestos y poses	12
3. Descripción del Trabajo	13
3.1. Modelo propuesto	13
3.2. Descripción del entorno	14

4. Conclusiones y Trabajo Futuro	19
4.1. Conclusiones	19
4.1.1. Conclusiones de la búsqueda de datasets	19
4.1.2. Conclusiones de la extracción de animaciones de bancos de animaciones	19
4.1.3. Conclusiones de la recolección de animaciones con usuarios	20
4.1.4. Conclusiones de la comparativa entre los modelos de IA	20
4.1.5. Conclusiones de la aplicación final	21
4.1.6. Limitaciones	21
4.2. Trabajo Futuro	21
Introduction	23
4.3. Motivation	23
4.4. Objectives	24
4.5. Work Plan	24
Conclusions and Future Work	27
4.6. Conclusions	27
4.6.1. Dataset Search Conclusions	27
4.6.2. Conclusions of the Animation Extraction Tool	27
4.6.3. Conclusions of the User Data Collection	28
4.6.4. Conclusions of the IA Models Comparison	28
4.6.5. Final Application Conclusions	28
4.6.6. Limitations	28
4.7. Future Work	29
Contribuciones Personales	31
Bibliografía	35
A. JSON Schema para los Behavior Trees generados	39
Glosario	43
Licencia de uso del documento	45
Licencia de uso del código fuente	47

Índice de figuras

1.1. Diagrama de Gantt del proyecto	3
2.1. Imágen del traje de captura de movimiento XSens	7
3.1. Diagrama de la herramienta propuesta	13
3.2. Entorno de desarrollo	14
4.1. Gantt chart of the project	25

Índice de tablas

2.1. Puntos de referencia del modelo YOLOv11-pose	12
---	----

Introducción

*“La revolución industrial y sus consecuencias han sido un
desastre para la raza humana”*
— Theodore Kaczynski

En el desarrollo de videojuegos es fundamental el diseño e implementación del comportamiento de los diferentes agentes presentes en el juego, tanto los [NPCs](#) como los enemigos, con el fin de crear un mundo inmersivo para el jugador. En los equipos de desarrollo es el diseñador quien especifica los comportamientos buscados, mientras que el programador tiene la tarea de implementar las acciones requeridas. Finalmente las acciones se deben juntar en un modelo ejecutor que sea capaces de procesarlas. Este trabajo se centra en el modelo de [Behavior Trees](#).

MOVER A MOTIVACIÓN: Gracias a los avances tecnológicos, los juegos son cada vez más grandes y con sistemas más complejos, lo que ha aumentado también la complejidad de los comportamientos generados, lo que también ha causado el incremento de la complejidad de los [Behavior Trees](#). Es por esto que el desarrollo de herramientas que permitan automatizar el montaje automático de estos modelos conseguirá reducir el tiempo del desarrollo de videojuegos.

1.1. Motivación

Como se ha mencionado en la introducción, el uso de la comunicación no verbal en entornos virtuales está limitado y prefijado por el diseño de éstos, limitando una parte fundamental de la forma de expresarse cotidiana de los humanos. Debido a esto y como podemos ver en artículos como [Neverova et al. \(2013\)](#) o [Herbert et al. \(2024\)](#) la detección de gestos es un campo de estudio que actualmente se está explorando para intentar resolver este problema. Por estas razones es interesante investigar formas de ampliar esta representación de la comunicación no verbal para que la comunicación entre usuarios y [NPCs](#) sea más natural.

Es por este motivo que este trabajo se inscribe dentro de la línea de investigación de herramientas que puedan reconocer gestos realizados por un usuario mediante un traje de captura de movimiento con el fin de crear entornos virtuales más inmersivos y naturales para los usuarios.

Por otra parte, es necesaria una gran cantidad de datos para crear modelos de [IA](#) que permitan crear las herramientas mencionadas. En el caso de este proyecto, los datos necesarios son una secuencia de coordenadas 3D de posiciones y rotaciones de los huesos del esqueleto a lo largo de las animaciones, por lo que los datos que se necesitan procesar tienen un tamaño relativamente grande. Es por esto que es necesario crear modelos complejos que procesen un número significativo de datos de entrada, pero como desarrollaremos a lo largo del trabajo, no ha sido posible encontrar datasets de este tipo de datos. Como consecuencia de este motivo, una gran aportación de este trabajo en el campo ha sido la creación de un dataset gracias a la extracción de animaciones de un banco especializado en estas y a la captura de movimientos de usuarios.

Vistos estos motivos, se puede concretar que, la carencia de métodos naturales de explotar la comunicación no verbal en entornos virtuales y la falta de datasets de animaciones han motivado los objetivos mencionados y que expandiremos a continuación.

1.2. Objetivos

El objetivo general de este proyecto es la implementación de un modelo de [IA](#) que, con baja latencia, permita identificar el gesto que se esté realizando con un traje de captura de movimiento en tiempo real, con la intención de mejorar la comunicación no verbal en entornos virtuales. Para cumplir el objetivo, se han propuesto las siguientes metas durante el desarrollo del trabajo:

1. Búsqueda de datasets de animaciones que se han considerado relevantes a la hora de comunicarse con un [NPC](#) en un videojuego. Estas animaciones han sido bailar, saludar, señalar, sentarse, pelear y correr.
2. Extracción automática de las animaciones mencionadas anteriormente de bancos de animaciones y posterior procesamiento.
3. Extracción de las animaciones mediante la captura de movimiento de un grupo heterogéneo de usuarios.
4. Implementación, entrenamiento y comparación de distintos modelos de [IA](#), tanto redes neuronales (LSTM, CNN y RNN) como modelos de clasificación clásicos (Random Forest)
5. Implementación de una aplicación final a modo de demostración para enseñar los resultados en [VR](#) para aumentar el grado de inmersión.

Como gran objetivo secundario y debido a la falta de datasets de animaciones se ha propuesto publicar los datasets resultantes, tanto de las animaciones artificiales como de las capturas de usuarios.

1.3. Plan de trabajo

El plan de trabajo consiste en varios pasos:

1. Búsqueda de un dataset: generar un dataset lo suficientemente grande con varios ejemplos de gestos como para poder entrenar de forma adecuada diferentes modelos. Este dataset debe estar compuesto en su mayoría por datos de usuarios reales para poder identificar gestos lo más naturales posibles.
2. Implementación de modelos de IA: implementación de varios modelos de IA para poder hacer una comparativa entre ellos y decidir cuál es el más adecuado teniendo en cuenta su velocidad de predicción y su precisión. La implementación de dichos modelos debe contener: entrenador, forma de sacar las estadísticas del modelo final, búsqueda de hiperparámetros, generador de matriz de confusión y debe ser acoplable a una interfaz de usuario común a todos los entrenamientos.
3. Desarrollo de una interfaz de usuario para el entrenamiento y monitorización de los distintos modelos: desarrollo de una interfaz web para que los usuarios con menos experiencia en programación e IA puedan entrenar modelos capaces de predecir los gestos que necesiten sin necesidad de modificar el código ni tener que saber como funciona internamente la base de código.
4. Desarrollo de una aplicación final: desarrollo de una aplicación para las Oculus Quest a forma de demo en la que se conecte al modelo elegido y se pueda ver en tiempo real su uso.

El desarrollo de las tareas y su planificación se puede ver en la figura 1.1.

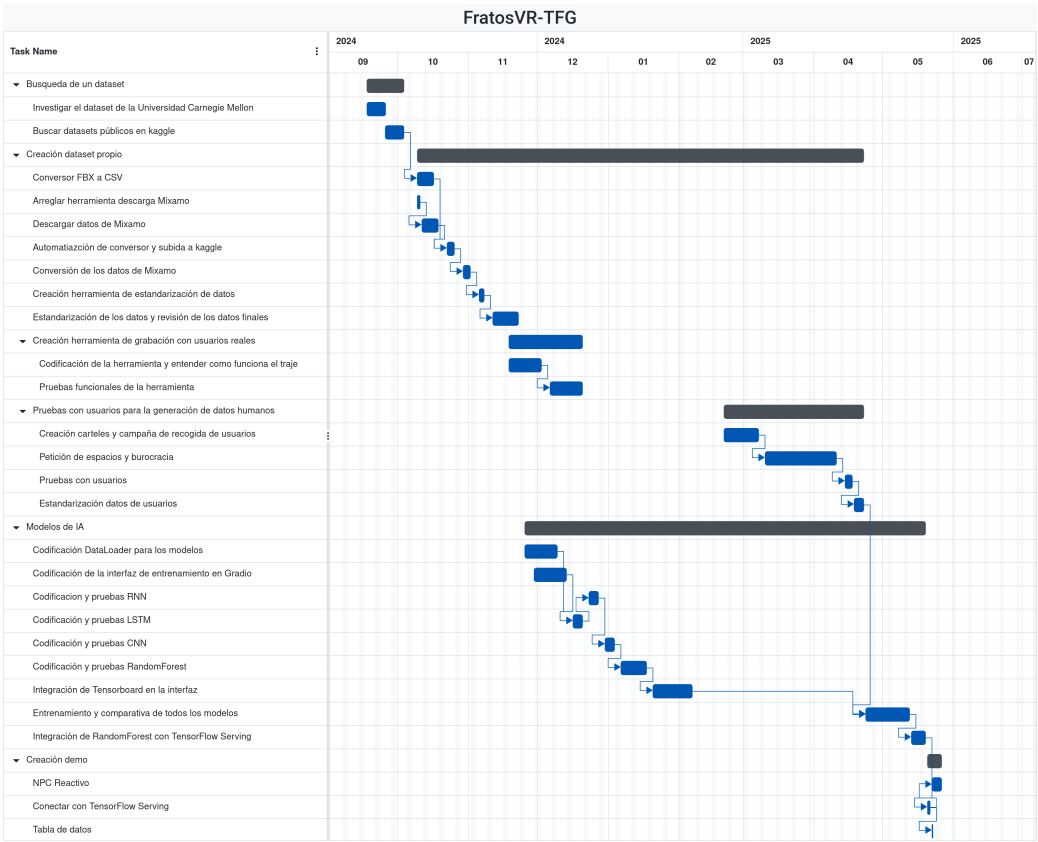


Figura 1.1: Diagrama de Gantt del proyecto

Capítulo 2

Estado de la Cuestión

En este capítulo se presenta el estado del arte de los distintos elementos que componen el trabajo. Estos campos son la comunicación no verbal en entornos virtuales, la captura de movimiento, distintos modelos de [IA](#), motores de videojuegos y el estado actual de los entornos virtuales.

2.1. Entornos virtuales

Como se puede ver en el artículo [Ellis \(1991\)](#), se empezó a hablar en la década de los 90 de entornos virtuales y el [metaverso](#) como nuevas formas de comunicarse con las personas. Estos entornos virtuales al principio eran muy sencillos, con pocas actividades que realizar y el chat escrito como única forma de comunicarse. Algún ejemplo de estos entornos virtuales eran el Second Life o el Habbo Hotel.

Para continuar hablando de los entornos virtuales es necesario introducir los conceptos de “inmersión” y “presencia”: dos términos que, a menudo se usan como sinónimos, pero son ligeramente distintos aunque están muy relacionados. Como lo define el artículo [Wilkinson et al. \(2021\)](#), la presencia es la calidad experimental en entornos virtuales; mientras que la inmersión está asociado con los aspectos técnicos de un sistema virtual que ayudan al usuario a potenciar la sensación de presencia. Estudios recientes como [Van Brakel et al. \(2023\)](#) demuestran como la presencia, tanto la nuestra propia como la social, son potenciadores del apoyo social percibido, lo que se asocia con el bienestar de los usuarios; mientras que el estudio [Pallavicini et al. \(2019\)](#) muestra que, aunque no haya mejoras performáticas entre jugar en entornos inmersivos ([VR](#)) y no inmersivos (un ordenador), los jugadores encontraron más emocionantes y con mayor presencia los entornos interactivos. Es por esto último que con la aparición de la [VR](#) se hicieron populares entornos virtuales focalizados en esta tecnología.

Con todo esto es natural que las empresas inviertan en tecnologías que aumenten la sensación de inmersión para lograr una mayor presencia. Empresas como Apple o Meta están apostando en la tecnología [VR](#), sacando recientemente las gafas Apple Vision Pro (2024) y Meta Quest 3 (2023); mejorando aspectos como la calidad de las cámaras, el sonido, el reconocimiento de gestos y su comodidad que pueden

amplificar la sensación de inmersión cuando se utilizan. Además, Meta está centrada en la idea de [metaverso](#) como un espacio digital en el que se puede socializar.

Con la idea de mejorar la inmersión en entornos virtuales se ideó este proyecto en el que, mediante la captura de movimiento, se puede usar la comunicación no verbal para expresar ciertas acciones y el entorno sea capaz de reconocer lo que estamos haciendo. Todo esto pensado para que se pueda usar en entornos de [VR](#) para llevar la inmersión al máximo

2.2. Comunicación no verbal en entornos virtuales

En los últimos años se han llevado a cabo distintos estudios sobre la comunicación no verbal en entornos virtuales. El estudio [Xenakis et al. \(2023\)](#) se centra en el concepto de presencia en entornos virtuales, enfatizando los roles de inmersión, contenido emocional y fidelidad de avatares en la mejora de la experiencia del usuario. El estudio resalta como el realismo de los avatares y sus comportamientos (movimiento y lenguaje corporal) ayudan a conseguir un mayor sentimiento de presencia social entre los usuarios. También se discute como estos elementos influyen en las dinámicas interpersonales y la percepción del usuario en entornos virtuales.

Estos roles discutidos en el estudio pueden extrapolarse a la experiencia de jugar a videojuegos. En el contexto de los videojuegos tradicionales, las opciones de comunicación no verbal son las dadas por los desarrolladores en forma de gestos, acciones dentro del juego (agacharse, girar su avatar o mover la cabeza al mover la cámara por ejemplo). Dentro de cada videojuego la comunidad va a intentar usar las opciones que se les proporcione para comunicarse entre ellos mismos de la mejor manera posible.

Un ejemplo claro de evolución de la comunicación no verbal en videojuegos y su importancia para los jugadores es el caso de *VR chat*. *VR chat* es un videojuego [MMORPG](#) de [VR](#) donde los jugadores pueden unirse a distintos mundos e interactuar con otros usuarios. Este juego no solo permite el uso de gafas de [VR](#), sino que también permite (de manera experimental) usar distintas formas de captura de movimiento de cuerpo completo ¹.

Los usuarios de esta aplicación han buscado distintas formas de conseguir una mayor expresividad en sus avatares para así poder entablar mejor conversaciones entre ellos o para poder expresarse como les gustaría. Algunos de estos métodos se describen en el siguiente apartado junto a métodos de captura de movimiento no orientados a videojuegos.

2.3. Captura de movimiento

La captura de movimiento es una técnica que permite digitalizar el movimiento de una persona. Esta técnica se usa sobre todo en el ámbito de la animación, el cine y los videojuegos. Para lograr esto se han desarrollado diferentes tipos de tecnologías:

¹Enlace a la documentación de *VR Chat* para su [SDK](#) de captura de movimiento de cuerpo completo [VRChat \(2024\)](#)

óptica, mecánica y magnética.

2.3.1. Captura de movimiento óptica

La captura de movimiento óptica consiste en el uso de cámaras para la captura. Según el artículo [Guerra-Filho \(2005\)](#) esta captura se puede dividir en monocular o multivista dependiendo del número de cámaras, y según si utilizan marcadores o no. En el mismo artículo explica el funcionamiento de la captura óptica multivista con el uso de marcadores, el cual se utiliza en gran medida en la industria audiovisual. Este tipo de captura óptica necesita al menos de un conjunto de cámaras sincronizadas (dos o más), un traje con marcadores en puntos significativos (articulaciones sobre todo) y un sistema de adquisición de varios vídeos simultáneos. La reconstrucción del movimiento se basa en la sincronía de las cámaras y la capacidad de detección de los mismos marcadores desde diferentes puntos de vista, haciendo que se pueda reconstruir en 3D los marcadores mediante la triangulación de los distintos puntos de vista.

Un ejemplo de la captura de movimiento óptica monocular y sin marcadores son los sistemas basados en cámaras 3D que utilizan los [IRs](#) para calcular la profundidad. Según el artículo [Zhang \(2012\)](#) la Kinect (Microsoft, 2010) utilizaba esta tecnología para ello. Consiste en un proyector de [IRs](#) cuyos rayos atraviesan una rejilla de difracción, convirtiéndose en un conjunto de puntos que, como se sabe la geometría relativa entre este proyector y una cámara de [IRs](#) que venía incorporada, se podía triangular la posición de un objeto haciendo coincidir un punto observado con uno de los puntos del proyector.

2.3.2. Captura de movimiento mecánica

La captura de movimiento mecánica se consigue mediante sensores inerciales. Según se puede ver en [Roetenberg et al. \(2009\)](#), el traje Xsens MVN es un ejemplo de ello. No utiliza cámaras o marcadores si no la combinación de giroscopios, acelerómetros y magnetómetros. Con los datos recibidos de estos sensores se puede estimar la rotación de los diferentes puntos, mientras que la posición se puede estimar mediante supuestos de una cadena cinemática. En la figura [2.1](#) se puede ver el traje visto desde la espalda.

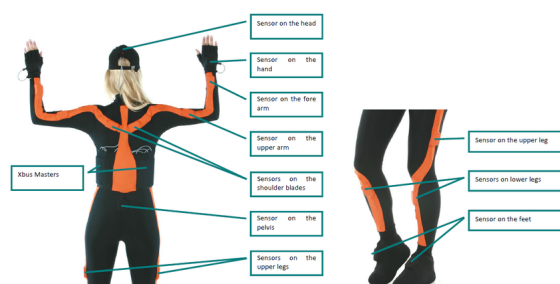


Figura 2.1: Imágen del traje de captura de movimiento Xsens ²

2.3.3. Captura de movimiento magnética

La captura magnética se consiste mediante sensores electromagnéticos. El artículo [Raab et al. \(1979\)](#) explica que la generación de valores en un campo electromagnético tridimensional es suficiente para determinar la posición y rotación de un objeto emisor frente a un sensor. Para ello se aplican transformaciones lineales a la excitación de la fuente y los vectores recibidos por el receptor para percibir pequeños cambios en los valores a determinar, los cuales se separan mediante combinaciones lineales de los vectores de salida del sensor y se utilizan para actualizar las coordenadas previas.

En esta última tecnología se basa el traje Perception Neuron 3. Este traje fue creado por Noitom, empresa fundada en el 2012 enfocada en el desarrollo de tecnología de captación de movimiento. El traje previamente mencionado se encuentra dentro de la serie de trajes “Perception Neuron”, los cuales son bandas de velcro a las que se pueden acoplar sensores electromagnéticos.

2.4. Motores de videojuegos

Un motor de videojuegos es un software que proporciona herramientas para desarrollar y ejecutar un videojuego. Estos motores están compuestos a su vez de otros motores más específicos que se encargan de funcionalidades básicas para un videojuego para que el desarrollador del juego no tenga que encargarse de ello. Algunos ejemplos de esto último son el motor de render, el de físicas o el de sonido.

Actualmente los motores de videojuegos más usados en el mercado son Unity, Unreal y Godot. Aunque el artículo [Holfeld \(2024\)](#) no se centre en un análisis del mercado en general, si contiene varias estadísticas sobre el uso de los principales motores de videojuegos en el mercado en 2023. Este artículo menciona que, de los juegos publicados en Steam ese año, un 60.04 % fueron hechos en Unity, un 16.42 % fueron hechos en Unreal y el 23.54 % restante fue hecho en motores propios de empresas (Source o CryEngine, por ejemplo).

2.4.1. Unity

Unity es un motor de videojuegos creado por Unity Technologies en 2005. A continuación se listan las principales herramientas que usa el motor:

1. Motor gráfico: Propio de Unity basado mayoritariamente en OpenGL (para depende de qué plataformas puede usar otro, como Direct3D para Windows).
2. Motor físico: PhysX, creado por la empresa Nvidia Corporation.
3. Motor de audio: Propio de Unity.
4. Lenguaje de scripting: C#.

²Fuente de la imagen: https://www.researchgate.net/figure/The-Xsens-MVN-full-body-motion-capture-suit-Picture-source-http-wwwxsenscom_fig1_233926874

5. Lenguaje de scripting de shaders: ShaderLab.

El modelo clásico de Unity se ha basado en la arquitectura Entidad-Componente (EC). Esta arquitectura deja a todo lo que hay en el juego (las entidades, llamadas en Unity “GameObjects”) como contenedores de componentes, que son los que le aportan funcionalidad a éstas. Este modelo supuso una mejoría ante la programación de videojuegos con objetos, ya que conseguía independizar comportamientos.

Desde 2022 mediante el paquete Unity DOTS (Data-Oriented Technology Stack)³ se pueden crear videojuegos siguiendo la arquitectura Entidad-Componente-Sistema (ECS). Esta arquitectura consiste en separar la lógica de los datos, ya que los datos se guardan en componentes mientras que la lógica la ejecutan los sistemas. ECS supone una mejoría a EC gracias a la separación mencionada, ya que mejora la escalabilidad del código y la modularidad.

2.4.2. Unreal

Unreal es un motor de videojuegos creado por Epic Games en 1998 con el lanzamiento de Unreal Engine 1. La versión estable actual es Unreal Engine 5.5, lanzada en noviembre de 2024. El código fuente de este motor está escrito en C++ y es accesible desde Github⁴. Las principales herramientas del motor son:

1. Motor gráfico: Propio de Unreal. Este cuenta con tecnologías para optimizar geometría (Nanite), sistema de iluminación dinámica (Lumen), y una técnica de escalado de imágenes (TSR)
2. Motor físico: Chaos Physics, desarrollado por Epic Games para mejorar la integración con las diferentes herramientas del motor.
3. Motor de audio: Propio de Epic Games llamado MetaSounds.
4. Lenguaje de scripting: C++ y Blueprints como scripting visual.
5. Lenguaje de scripting de shaders: High-Level Shading Language (HLSL) y Material Editor como forma de abstracción, permitiendo crear materiales con nodos visuales.

Unreal utiliza una arquitectura orientada a clases jerárquicas más cercana a la programación orientada a objetos con alguna integración con componentes. Los objetos del mundo, llamados “actores”, heredan de la clase AActor y son los componentes quienes le dan alguna de las funcionalidades (mallas, físicas o cámara por ejemplo). Las clases controlables (tanto por el jugador como por IA) son entidades de tipo “Pawn” (clase APawn), de la cual hereda los objetos de tipo “Character” (clase ACharacter), los cuales tienen un movimiento humanoide incorporado. Para introducir movimiento a los objetos Pawn es necesario hacerlo mediante objetos de tipo controller (APlayerController para entrada de input y AAIController para IA).

³Enlace a la documentación del paquete DOTS: <https://docs.unity3d.com/Packages/com.unity.entities@1.3/manual/index.html>

⁴Enlace a la página de Github de Epic: <https://github.com/epicgames>

2.5. Modelos de IA

La inteligencia artificial ha sido un tópico de desarrollo e investigación en los últimos años. Aunque el desarrollo en los últimos años se ha centrado sobre todo en el ámbito de las IAs generativas, este estudio se va a centrar sobre todo en el uso de redes neuronales e IAs de clasificación más tradicionales.

2.5.1. Tecnologías de IA

En la actualidad, el mundo de la IA está centrado en el desarrollo en python mediante el uso de librerías programadas en C++. Las librerías más comunes son Tensorflow, Pytorch y scikit-learn.

Tensorflow tiene un enfoque más orientado a la producción y despliegue de modelos de IA a un nivel empresarial, pytorch es más usado en el ámbito académico y de investigación por su capacidad de rápido prototipado. Scikit-learn se usa más para enseñar los principios de la IA y desarrollar modelos más ligeros y más tradicionales.

Estas librerías se usan en conjunto con Numpy(Harris et al. (2020)) y Pandas(pandas development team (2020), Wes McKinney (2010)) para conseguir unos cálculos más efectivos sobre la gran cantidad de datos que se necesitan en el ciclo de vida de un modelo de IA.

2.5.2. Librerías de IA

En el apartado anterior ya se han nombrado distintas librerías de IA que se usan en la actualidad. En este apartado se van a describir las más relevantes para el desarrollo de este trabajo.

2.5.2.1. Tensorflow

Tensorflow(Abadi et al. (2015)) es una librería de IA desarrollada y mantenida por el equipo de Google Brain desde 2015. Esta librería permite el desarrollo de modelos de IA usando CPUs, GPUs y TPUs. Tensorflow es una plataforma de código abierto que tiene un ecosistema de herramientas y librerías que permite el desarrollo e investigación de modelos de IA. Tensorflow tiene un enfoque más orientado a la producción y despliegue de modelos de IA a un nivel empresarial, por lo que es más común ver su uso en empresas que en el ámbito académico. Un ejemplo de este enfoque es la presencia de la herramienta Tensorflow Serving Olston et al. (2017), que permite el despliegado de modelos de Tensorflow (o compatibles con el ecosistema) de una forma sencilla.

2.5.2.2. PyTorch

PyTorch(Ansel et al. (2024)) fue desarrolladapor Facebook AI Research en 2026, se hizo de código abierto en 2017 y en 2022 se unió a la Linux Foundation. Esta librería se centra en la idea de que la búsqueda de usabilidad y velocidad son compatibles. Pytorch permite el desarrollo de modelos de IA al igual que Tensorflow, pero su

modelo de código y desarrollo ha hecho que se use más para prototipado rápido y desarrollo de proyectos dinámicos. Pytorch permite la integración con otras librerías como NumPy, SciPy y Cython, lo que permite un desarrollo más rápido y sencillo de modelos de IA.

2.5.2.3. Scikit-learn

Scikit-learn(Pedregosa et al. (2011)) es una librería de IA desarrollada en Python que se centra en el aprendizaje automático. Esta librería se basa en NumPy, SciPy y matplotlib, lo que permite un desarrollo más sencillo de modelos de IA. Scikit-learn es una librería más ligera y más tradicional que Tensorflow o Pytorch, por lo que es más común ver su uso en el ámbito académico y de investigación. Scikit-learn se centra más en el análisis de datos predictivo y también es de código abierto como las anteriores.

2.5.2.4. YDF

YDF (Guillame-Bert et al. (2023)) es una librería para entrenar, evaluar, interpretar y desplegar Random Forest, gradient boosted decision trees, CART y isolation forest. Esta librería esta intregrada con Tensorflow y keras, centra su desarrollo en la optimización de árboles de decisión y en hacer una API concisa y moderna.

2.5.3. Modelos de reconocimiento de gestos y poses

Muchos modelos de IA se han desarrollado para la detección de gestos y poses. Estos modelos pueden usar imágenes, videos, datos de sensores o esqueletos formados por puntos de referencia. En este apartado se van a destacar los más relevantes para el resto del trabajo.

2.5.3.1. YoLoV11

YoLoV11(Jocher et al. (2023)) es un conjunto de modelos YOLO, entre los que se encuentra un modelo de detección de poses. Por defecto este modelo usa 17 puntos de referencia (presentados en la tabla 2.1) y usa imágenes de 640x640 píxeles como entrada. En la página del modelo de pose⁵, Ultralytics incluye los distintos modelos de pose y su rendimiento y requerimientos. Este modelo se ofrece preentrenado en distintos tamaños con la opción de re entrenarse usando un dataset con el formato especificado por Ultralytics⁶

⁵Página de YOLOv11-pose: <https://docs.ultralytics.com/es/tasks/pose/>

⁶Página de especificación del formato del dataset: <https://docs.ultralytics.com/es/datasets/pose/>

Tabla 2.1: Puntos de referencia del modelo YOLOv11-pose

Nariz
Ojo izquierdo
Ojo derecho
Oreja izquierda
Oreja derecha
Hombro izquierdo
Hombro derecho
Codo izquierdo
Codo derecho
Muñeca izquierda
Muñeca derecha
Cadera izquierda
Cadera derecha
Rodilla izquierda
Rodilla derecha
Tobillo izquierdo
Tobillo derecho

2.5.3.2. Modelos para la detección del balance de personas andando

En el artículo [Ma et al. \(2023\)](#) se presenta una comparativa de modelos para la detección del balance de personas andando en distintos niveles de valance y con distintas restricciones con un traje usado. Este artículo presenta una generación de esqueletos en 3D a partir de una kinect y realiza una comparativa de distintos modelos tradicionales y neuronales para ver su efectividad. En las conclusiones podemos ver como, de los distintos modelos utilizados, el que mejor resultados da es un [DCNN](#) que los investigadores proponen. Otros modelos de la comparativa incluyen el [Random Forest](#), el [SVM](#), [LSTM](#) y Naive Bayes en términos de modelos tradicionales. Otra comparativa se realiza con otros modelos basados en [CNNs](#) como AlexNet, VGGNet y ResNet-18.

2.5.3.3. Otros modelos de detección de gestos y poses

En los últimos años se han desarrollado modelos generales de detección de gestos y poses. Estos modelos han tenido distintos enfoques, desde el uso de gafas de [VR](#) para hacer detección de gestos simples con las manos para utilizarlos como método de entrada (Meta Quest 2 y 3 por ejemplo), a modelos de seguimiento de manos y distintos modelos de reconocimiento de Google con Gemini.

Capítulo 3

Descripción del Trabajo

3.1. Modelo propuesto

Esta herramienta debería ser de fácil uso para los diseñadores y programadores, por lo que hemos propuesto un modelo en el que, sin salir del proyecto de Unreal Engine, puedan generar los Behavior Trees de forma sencilla.

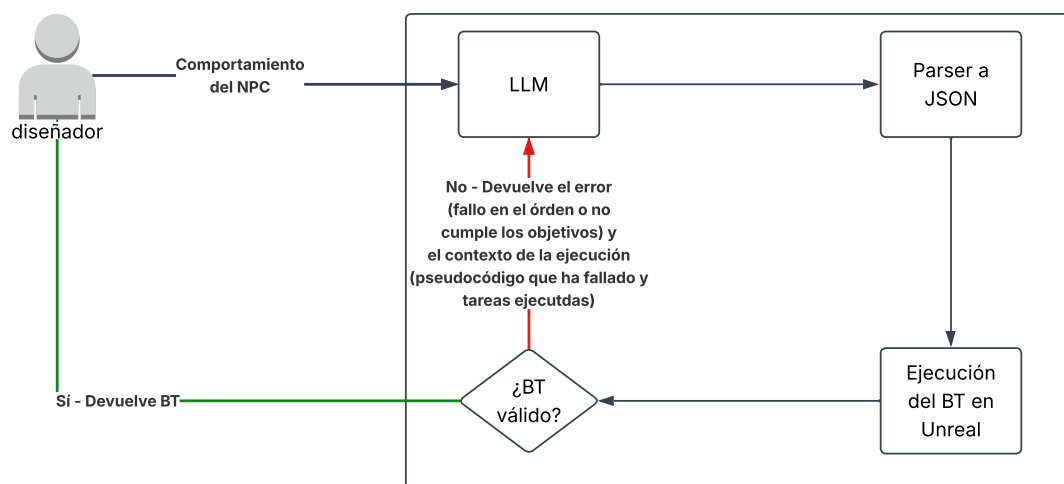


Figura 3.1: Diagrama donde se visualiza el flujo de la herramienta propuesta.

En la figura 3.1 se puede ver el flujo de la herramienta, donde el diseñador describe el comportamiento que busca para un NPC y, de forma opaca para éste, un LLM genera el Behavior Tree, el cual deberá ser parseado a formato JSON. Una vez creado el BT, se devuelve el control a Unreal Engine, donde se ejecutará de forma automática para comprobar que es un Behavior Tree válido. En caso de ser así se termina la ejecución de la herramienta, pero en caso contrario se reinicia su ejecución, donde se le pasa al LLM el error del Behavior Tree generado para volver a intentarlo.

En las siguientes secciones de este capítulo se explicará el entorno de pruebas

desarrollado para este trabajo, así como la implementación de la generación de los [Behavior Trees](#), la validación de éstos y las pruebas realizadas.

3.2. Descripción del entorno

Para poder desarrollar la herramienta se han creado unos entornos de pruebas basado en Unreal Engine 5.7. Son estos entornos los que se van a comunicar con el agente [LLM](#) de Python.

Estos entornos, de los que se pueden ver un ejemplo en la figura 3.2, constan de un agente (el [NPC](#) que va a seguir el comportamiento buscado) cuyo controlador es una clase que hereda de la clase de Unreal Engine “AIController”. Esta clase contiene los componentes de [Blackboard](#) y [Behavior Tree](#) necesarios para la futura ejecución de un [Behavior Tree](#). En la misma figura se puede apreciar la interfaz en la que el usuario debe introducir la información necesaria para la generación del [Behavior Tree](#): la descripción del comportamiento buscado, el nombre con el que se va a guardar el [Behavior Tree](#) generado, las entradas de la [Blackboard](#), el agente que va a ejecutar el [Behavior Tree](#) generado y, opcionalmente, el nombre del archivo de validación del [Behavior Tree](#) generado. Esta información la debe contener un actor, llamado en la captura “BTGeneratorManager”, que tenga un componente llamado “BTGenerator”. Este componente de actor es el encargado de lanzar el proceso de Python con la información que el usuario haya puesto, y el que comienza la ejecución de las pruebas del [Behavior Tree](#) generado. Para comenzar la ejecución se debe pulsar el botón “GenerateBT” que se aprecia en la captura.

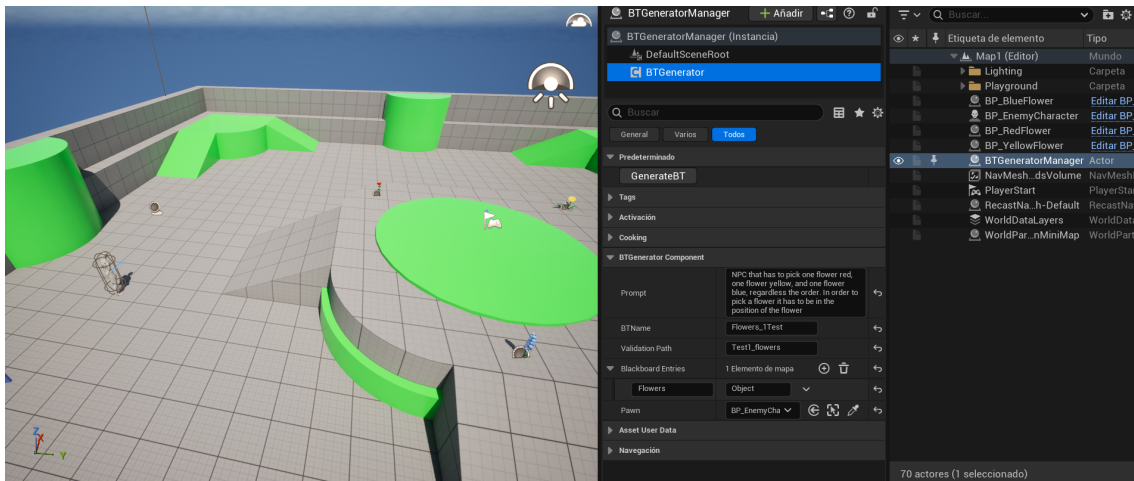


Figura 3.2: Captura de pantalla de uno de los entornos creados para el desarrollo de la herramienta.

Junto con este agente se han preparado una serie de tareas (los cuales servirán como nodos terminales de los árboles) y condicionales que sirven como base para poder crear unos comportamientos de prueba. Estas tareas y condicionales son introducidos en una clase “BTFactory”, la cual es un [Abstract Factory](#) implementado para la construcción de los [Behavior Trees](#). Esta clase, la cual se inicializa desde el

componente “BTGeneratorComponent”, contiene tres diccionarios que guardan las tareas, las tareas que necesitan una variable de la [Blackboard](#) y los condicionales. Estos diccionarios reciben como clave un string, que es la ID de la tarea o el condicional, y como valor una función (implementada en la propia clase) que crea el objeto que se ha pedido. Además esta clase tiene otro diccionario que contiene la descripción de cada tarea y condicional, la cual es fundamental para la futura generación de los [Behavior Trees](#). Para registrar una nueva clase en esta factoría se debe llamar a las funciones “RegisterTask”, “RegisterBlackboardTask” o “RegisterDecorator”, las cuales son funciones template en las que se les debe poner la clase que se desea ser introducida y pasar como argumentos el ID que se desee poner y la descripción de lo que realiza la tarea o lo que revisa el condicional. Además, para las clases que requieran una variable de la [Blackboard](#) es recomendable introducir qué tipo de variable se requiere.

Las tareas, junto con sus descripciones en español (en el trabajo realizado las descripciones han sido escritas en inglés para respetar el idioma del prompt) que se han creado para estas pruebas son las siguientes:

- **RotateToFaceBBEntry**: Rotar hasta encarar objetivo. El objetivo tiene que ser de tipo “Objeto”.
- **ChasePlayer**: Perseguir al jugador.
- **MoveTo**: Mover a la posición u objeto de destino. La variable objetivo tiene que ser de tipo “Objeto” o “Vector3”
- **FindRandomPatrol**: Elegir un punto aleatorio del mapa y guardarlo como objetivo.
- **Wait**: No hacer nada.
- **PickRedFlower**: Coger una flor roja.
- **PickBlueFlower**: Coger una flor azul.
- **PickYellowFlower**: Coger una flor amarilla.
- **PickBlackFlower**: Coger una flor negra.
- **ChooseRedFlower**: Elegir una flor roja como objetivo.
- **ChooseBlueFlower**: Elegir una flor azul como objetivo.
- **ChooseYellowFlower**: Elegir una flor amarilla como objetivo.
- **ChooseBlackFlower**: Elegir una flor negra como objetivo.
- **Attack**: Atacar si hay algún objetivo cerca.
- **SelectWaypoint**: Elegir un punto de ruta aleatorio y establecerlo como objetivo. El objetivo debe ser de tipo “Objeto”

Los condicionales implementados, junto con sus descripciones en español (al igual que las tareas, las descripciones en el trabajo están en inglés), son los siguientes:

- **HasLineOfSight?**: Ciertó si el jugador está en línea de visión, falso si no. La variable que verifica tiene que ser de tipo “Bool”
- **IsItNear?**: Ciertó si la distancia entre el ejecutor y el jugador es menor o igual que un umbral prefijado, falso en caso contrario. El objetivo tiene que ser de tipo “Objeto”.

Las tareas y condicionales que necesiten una variable de la [Blackboard](#) necesitan seleccionarla desde el objeto de la [Blackboard](#) (llamado en Unreal “BlackboardData”) a través de una variable que tienen los condicionales y las tareas que hereden de la clase de Unreal “BTTask_BlackboardBase”, que son las tareas que requieren de una variable. Sin embargo esta variable es protegida, por lo que se ha creado una interfaz intermedia llamada “BaseTask” (de la que heredan todas las tareas que requieran una variable) que contiene las funciones necesarias para acceder a esta variable protegida que permite seleccionar la variable deseada de la [Blackboard](#).

El número de terminales y decoradores disponibles en esta implementación es reducido, pero sirve como sistema de prueba para la generación y la propuesta de evaluación.

Para la generación del JSON se ha seguido una definición de JSON Schema (Draft 2020-12) que delimita la estructura de la generación del JSON para adaptarlo a las necesidades del trabajo. Este JSON Schema consta de dos partes: el [Blackboard](#) y el [Behavior Tree](#). El [Blackboard](#) es una lista de objetos JSON que contienen solo un par clave-valor, donde la clave es el nombre de la variable y el valor es el tipo de la variable (Bool, Int, Float, Vector, Object o String). El [Behavior Tree](#) consta de un nodo (la raíz) que debe contener el tipo de nodo (Task, Sequence, Selector o Parallel), una lista de decoradores (en el caso de no tener se debe quedar vacía), la cual consiste en objetos JSON que debe contener como clave el nombre del decorador y como valor la variable de la [Blackboard](#) que va a usar, y una lista de nodos hijos. Estos nodos hijos siguen la misma estructura que la raíz. Como propiedades adicionales, puede contener una lista de nombres de variables de la [Blackboard](#) que vaya a necesitar, y, en el caso de ser una tarea, debe tener el ID de ésta. Este esquema se puede encontrar en el apéndice [A](#)

Para cuando el modelo haya generado el JSON con el [Behavior Tree](#), la clase “BTGeneratorComponent” entra en el modo de juego. Es en este momento donde el [NPC](#), desde su función “BeginPlay” (función de Unreal que es llamada cuando se inicia el juego), coge el nombre del archivo JSON generado para traducirlo a la lógica de Unreal. Esta traducción la hace la clase “BTConstructor”, la cual es una clase que sigue el patrón [Singleton](#) que es inicializada desde la clase “BTGeneratorComponent”. Para la traducción se le pasa a la clase el nombre del archivo y el componente de [Blackboard](#) que debe contener el [NPC](#) mediante el método “CreateBT”. Este método crea un [Behavior Tree](#) (clase “BehaviorTree”) y un [Blackboard](#) (clase “BlackboardData”) que serán rellenados con la información del JSON.

Lo primero que se traduce es el [Blackboard](#) gracias al método “CreateBlackboardAsset”, el cual recorre toda la lista de variables para, dependiendo del tipo de ésta,

crear un nuevo objeto de Unreal de tipo “BlackboardKeyType_XX” (donde las XX es el tipo de la variable). Éstas se guardan dentro del campo “KeyType” de una variable creada de tipo “BlackboardEntry”, que será agregada al campo “Keys” de la variable de tipo “BlackboardData”. Cuando ya se han creado todas las variables de la [Blackboard](#) se inicializa mediante el método “InitializeBlackboard” del componente “BlackboardComponent”.

Finalmente se crea el [Behavior Tree](#) mediante el método recursivo “CreateNode”. Inicialmente crea un objeto de tipo “FBTCompositeChild” en el que se guardará y devolverá el nodo que se le haya pasado como parámetro (inicialmente el nodo raíz). Si no es de tipo “Task” se crea un objeto de tipo “BTCompositeNode”, del cual heredan los nodos compuestos (Selector, Sequence y Parallel) para, posteriormente y de forma recursiva, incializar los nodos hijos comprendidos dentro de la lista “Nodes” y establecérselos como sus hijos. Si por el contrario es de tipo “Task” se le pide a la clase “BTFactory” una nueva instancia de esa tarea (si no existe esta tarea se devuelve un nodo vacío). Además, si necesita alguna variable de la [Blackboard](#), se llama a las funciones necesarias de éstas tareas para establecérsela. Finalmente, tanto como para las tareas como para los nodos compuestos, se recorren sus decoradores para inicializarse de la misma forma que las tareas: pidiéndole a “BTFactory” una instancia de esa clase y estableciendo las variables de la [Blackboard](#) necesarias.

Conclusiones y Trabajo Futuro

4.1. Conclusiones

El objetivo de este estudio era la creación de un modelo de [IA](#) que pudiera detectar una serie de gestos de usuarios con un traje de captura de movimiento. Para ello, se buscaron datasets de gestos que concordaran con las restricciones del trabajo. Al no encontrar nada fácilmente adaptable o con los gestos necesarios, se creó un dataset de animaciones, con datos tanto generados por ordenador como recopilados de usuarios reales.

Las animaciones generadas por ordenador se convirtieron a [CSV](#) por una herramienta creada por los autores del trabajo, al igual que las recolectadas por usuarios reales se hicieron con otra herramienta de los mismos creadores.

Tras los entrenamientos de los distintos modelos de [IA](#), se llegó a la conclusión de que el [Random Forest](#) es el mejor modelo por diferentes razones. Tras esto, se creó una aplicación a modo de demostración para mostrar el resultado final.

Las conclusiones de cada una de las partes del trabajo se presentan en las siguientes secciones de manera más detalladas.

4.1.1. Conclusiones de la búsqueda de datasets

Tras la búsqueda de datasets de animaciones se ha llegado a la conclusión de que no existen datasets públicos suficientes para el objetivo de este estudio.

Estas conclusiones nos llevaron a la necesidad de crear un dataset propio recabando animaciones de bancos de animaciones y mediante pruebas con usuarios.

4.1.2. Conclusiones de la extracción de animaciones de bancos de animaciones

Para la conversión de animaciones de Mixamo se ha creado una herramienta para hacerlo de forma automática y se ha modificado otra herramienta para hacer una descarga en masa de estas animaciones.

La herramienta de conversión de gestos cumple su objetivo completamente, ya

que tan solo indicando el nombre de la carpeta que contiene los gestos descargados coge los gestos, se procesan en Unity y se suben a Kaggle de forma automática. Además la herramienta es independiente al número de gestos que se quieren convertir y a su vez del número de animaciones que hay de esos gestos, consiguiendo una gran escalabilidad de esta forma haciendo posible la introducción de nuevos gestos sin tener que cambiar nada de la herramienta.

4.1.3. Conclusiones de la recolección de animaciones con usuarios

Para la recolección de datos con usuarios se anunció de forma insistente en las redes sociales de los autores del estudio así como por las distintas facultades de la Universidad Complutense de Madrid. Los anuncios cumplieron su objetivo, consiguiendo 75 respuestas de los cuales se presentaron 65 participantes, consiguiendo un 86,67 % de asistencia.

Como se puede ver en las figuras del apéndice ?? el grupo de participantes, con excepción del género no binario, es heterogéneo en cuanto al género con una pequeña diferencia de porcentaje entre hombres y mujeres. Donde se puede ver mayor diferencia es entre la mano dominante de los usuarios, siendo los zurdos tan solo un 6.15 % de la población total del grupo.

Gracias a la prueba se obtuvieron 975 animaciones (195 animaciones de saludar, señalar, sentarse, pelear y correr).

4.1.4. Conclusiones de la comparativa entre los modelos de IA

En cuanto a los modelos de IA, se ha visto que los modelos de redes neuronales no han obtenido los resultados esperados, siendo esto seguramente por una mezcla de falta de datos y simplificación de las redes neuronales para asegurar un tiempo de entrenamiento menor y un computo más rápido a la hora de realizar inferencias.

Sin embargo, el [Random Forest](#) ha sido el que ha obtenido mejores resultados, siendo el modelo más rápido (tanto en inferencia como en entrenamiento), el modelo más ligero y el modelo más adaptable. Este modelo se podría usar en sistemas con recursos limitados como pueden ser gafas de [VR](#) independientes como las Meta Quest o en ordenadores más potentes, e incluso en ecosistemas de IA por su posible traducción a TensorFlow ya implementada.

En el análisis de los resultados del [Random Forest](#) se ha visto que el modelo le da una gran importancia a la coordenada y de la pierna izquierda, a la coordenada y del pie izquierdo en el segundo intervalo de tiempo. Esto puede indicar que los gestos dependen mucho de la posición de las piernas para discriminar si se está corriendo, bailando, saludando o señalando.

También se puede ver una ligera confusión entre los gestos de pelea y saludar. Esto puede ser debido a los movimientos rápidos de las manos en ambos tipos de gesto, que al no tener dedos por limitación del traje, puede dar lugar a confusión entre un puñetazo y un saludo.

4.1.5. Conclusiones de la aplicación final

La aplicación final es una demo que se conecta a un servidor para mostrar los resultados.

La parte negativa de esta aplicación es la forma de conectarse con el servidor en el que está el modelo, ya que hace peticiones web al servidor de Tensor Serving directamente, lo que lo hace dependiente de éste.

Finalmente la aplicación final cumple su propósito de enseñar al usuario la animación predicha con una latencia mínima.

4.1.6. Limitaciones

Este estudio ha tenido una serie de limitaciones, desde escasez de datasets públicos, desbalanceo de tipos de animaciones, falta de más datos de usuarios reales, problemas de entrenamiento con los modelos de redes neuronales y problemas de falta de datos para estos mismos modelos.

La limitación de escasez de datasets se debe no solo a la falta en número de datasets, sino también a la falta de gestos referentes a comunicación no verbal, habiendo algunos sobre deportes o temáticas muy concretas y pocos de carácter general. Otra limitación de estos datasets ha sido la facilidad de adecuación o conversión a un formato compatible con el traje de captura de movimiento usado en el estudio.

En cuanto al desbalanceo de los tipos de animaciones, la problemática viene dada por la duración de las animaciones de baile de Mixamo. Al seguir el proceso de estandarizado, las distintas animaciones de baile se multiplican en un número bastante mayor que los del resto de tipos. Esto no se solucionó con los datos recopilados de usuarios reales, ya que aún que se obtuvo un gran número de animaciones nuevas, tras la estandarización las de baile seguían prevaleciendo por bastante. Esto se puede ver en las figuras ?? y ??.

Los modelos de redes neuronales no han dado los resultados esperados, esto puede ser por la falta de datos, que aunque haya un número que a simple vista parece suficiente no lo es para entrenar redes neuronales, por las limitaciones de hardware para entrenar modelos más complejos y por los tiempos requeridos para la realización de este estudio no ser lo suficientemente largos como para llevar a cabo entrenamientos más extensos y con mayor búsqueda de hiperparámetros.

Todas estas limitaciones, han llevado al siguiente plan de trabajo a futuro.

4.2. Trabajo Futuro

El trabajo a futuro de este estudio se puede centrar en varios esfuerzos:

- Mejorar el dataset, incluyendo gente con diversidad funcional, aumentando el número de muestras por gesto, teniendo en cuenta los dedos de las manos y añadiendo nuevas categorías. Esto no solo puede llegar a mejorar el rendimiento de modelos de redes neuronales, sino que puede dotar de mayores diferencias a los modelos para poder generalizar mejor.

- Investigar otros modelos de IA que puedan mejorar la precisión de la clasificación y hacer clasificación no solo de gestos, sino también de emociones. Esto podría ayudar a que los NPCs puedan reaccionar de una manera más natural a los gestos del usuario, haciendo la interacción más inmersiva.
- Estandarizar el servidor para que pueda ser usado con otras tecnologías de IA y no solo con TensorFlow. Esto permitiría integrar tecnologías que vayan surgiendo en el futuro a aplicaciones ya existentes.
- Implementar otro modelo que no solo tenga en cuenta el gesto predicho actual sino también todo el contexto para poder avanzar en el uso de la comunicación no verbal con NPCs.
- Evaluación del resultado final con usuarios.

Introduction

In recent years, the advancement of immersive technologies such as [VR](#) has brought about a significant change in how users interact in digital environments. Major companies like Meta and Apple have invested in the development of hardware for these technologies, launching devices like the Meta Quest 3 and Apple Vision Pro, and creating social interaction platforms in the [metaverse](#) with the development of Meta Horizon Worlds. This technology has opened new opportunities to explore more natural modes of communication in virtual spaces through non-verbal communication.

Non-verbal communication refers to the transmission of messages without the use of words, relying instead on images and gestures. In our daily lives, it plays a crucial role in human interactions, yet its representation in virtual environments is often limited and dependent on predefined actions such as animations or "emotes" already integrated into the game. Therefore, the development of tools that can capture and interpret these gestures in real-time represents a significant step towards creating more expressive and realistic virtual environments.

4.3. Motivation

As mentioned in the introduction, the use of non-verbal communication in virtual environments is limited and predefined by their design, restricting a fundamental part of everyday human expression. Due to this, and as seen in articles like [Neverova et al. \(2013\)](#) or [Herbert et al. \(2024\)](#), gesture detection is a field of study currently being explored to address this issue. For these reasons, it is interesting to investigate ways to expand this representation of non-verbal communication so that interaction between users and [NPCs](#) becomes more natural.

This work is therefore part of the research line focused on developing tools that can recognize gestures performed by a user using a motion capture suit, with the aim of creating more immersive and natural virtual environments for users. On the other hand, a large amount of data is required to create [AI](#) models that enable the development of the mentioned tools. In the case of this project, the necessary data consists of a sequence of 3D coordinates representing the positions and rotations of the skeleton's bones throughout the animations, resulting in relatively large data sizes. This necessitates the creation of complex models capable of processing a

significant amount of input data. However, as we will discuss throughout the work, it has not been possible to find datasets containing this type of data. As a result, a significant contribution of this work to the field has been the creation of a dataset through the extraction of animations from a specialized bank and the motion capture of users.

Given these reasons, it can be concluded that the lack of natural methods to leverage non-verbal communication in virtual environments and the scarcity of animation datasets have motivated the objectives mentioned, which we will expand upon below.

4.4. Objectives

The general objective of this project is to implement an [AI](#) model that, with low latency, allows real-time identification of gestures performed with a motion capture suit, aiming to enhance non-verbal communication in virtual environments. To achieve this objective, the following goals have been proposed during the development of the work:

1. Search for animation datasets that are considered relevant for communicating with an [NPC](#) in a video game. These animations include dancing, greeting, pointing, sitting, fighting, and running.
2. Automatic extraction of the aforementioned animations from animation banks and subsequent processing.
3. Extraction of animations through motion capture from a heterogeneous group of users.
4. Implementation, training, and comparison of different [AI](#) models, including neural networks (LSTM, CNN, and RNN) as well as classical classification models (Random Forest).
5. Development of a final application as a demonstration to showcase the results in [VR](#) to enhance immersion.

As a significant secondary objective, due to the lack of animation datasets, it has been proposed to publish the resulting datasets, both from artificial animations and user captures.

4.5. Work Plan

The work plan consists of several steps:

1. Search for a dataset: generate a sufficiently large dataset with multiple examples of gestures to adequately train different models. This dataset should primarily consist of data from real users to identify the most natural gestures possible.

- 2. Implementation of AI models: implement various AI models to compare them and determine which one is most suitable based on prediction speed and accuracy. The implementation of these models should include: training, a way to extract final model statistics, hyperparameter search, confusion matrix generator, and compatibility with a common user interface for all trainings.
- 3. Development of a user interface for training and monitoring different models: develop a web interface so that users with less programming and AI experience can train models capable of predicting the gestures they need without modifying the code or needing to understand the internal workings of the codebase.
- 4. Development of a final application: create an application for Oculus Quest as a demo that connects to the chosen model and allows real-time usage demonstration.

The development of tasks and their planning can be seen in Figure 4.1.

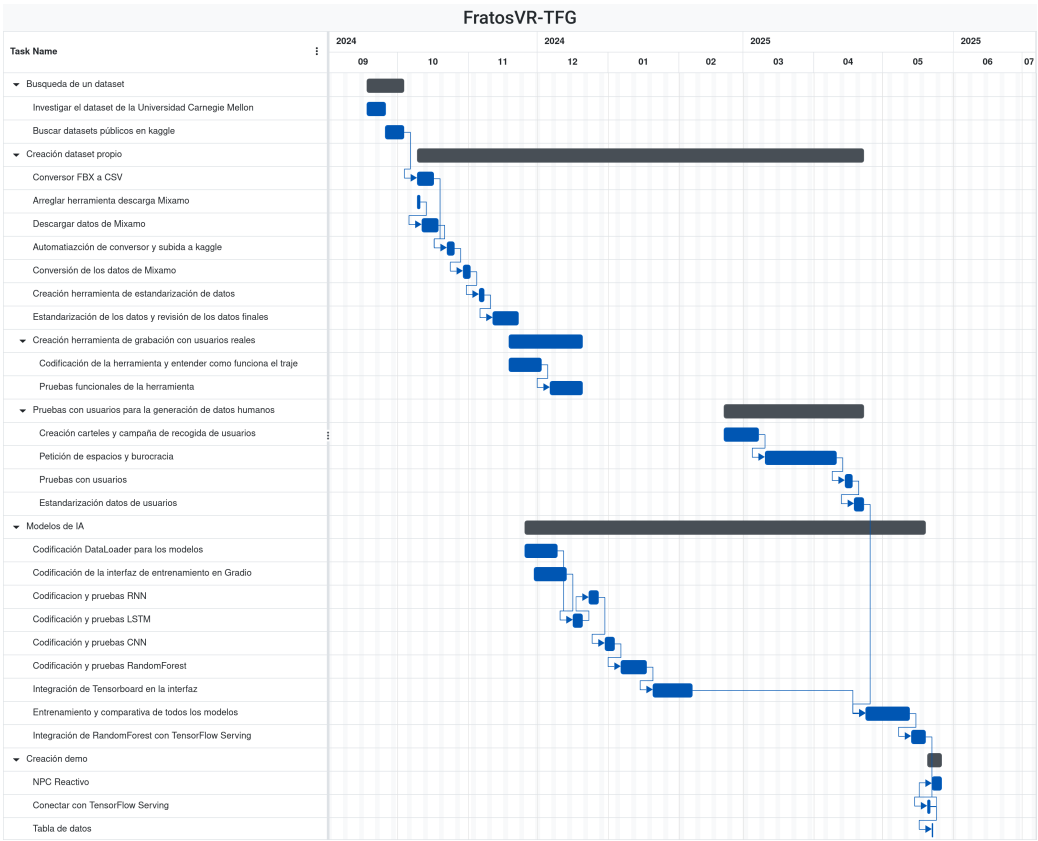


Figure 4.1: Gantt chart of the project

Conclusions and Future Work

4.6. Conclusions

The objective of this study was to create an [AI](#) model capable of detecting a series of gestures from users wearing a motion capture suit. To achieve this, datasets of gestures that matched the study's constraints were sought. Since no easily adaptable datasets with the required gestures were found, an animation dataset was created, consisting of both computer-generated data and data collected from real users. The computer-generated animations were converted to [CSV](#) format using a tool created by the authors of the study, while the animations collected from real users were processed with another tool developed by the same authors. After training various [AI](#) models, it was concluded that the [Random Forest](#) model is the best choice for several reasons. Following this, a demonstration application was created to showcase the final results.

The conclusions of each part of the work are presented in the following sections in more detail.

4.6.1. Dataset Search Conclusions

After the search for animation datasets, it was concluded that there are not enough public datasets available for the objective of this study.

These conclusions led us to the need to create our own dataset by collecting animations from animation banks and through user trials.

4.6.2. Conclusions of the Animation Extraction Tool

The gesture conversion tool fully meets its objective, as it only requires specifying the name of the folder containing the downloaded gestures. It automatically processes the gestures in Unity and uploads them to Kaggle. Additionally, the tool is independent of the number of gestures to be converted and the number of animations available for those gestures, achieving great scalability. This allows for the introduction of new gestures without requiring any changes to the tool.

4.6.3. Conclusions of the User Data Collection

As can be seen in the figures in Appendix ??, the participant group—except for the non-binary gender category—is heterogeneous in terms of gender, with only a small percentage difference between men and women. The most notable difference can be seen in users' dominant hand, with left-handed individuals making up only 6.15% of the group's total population.

Thanks to the test, a total of 975 animations were obtained (195 animations each for waving, pointing, sitting, fighting, and running).

4.6.4. Conclusions of the IA Models Comparison

In terms of the AI models, it has been observed that neural network models did not achieve the expected results, likely due to a combination of insufficient data and simplification of the neural networks to ensure shorter training times and faster inference computations.

However, the Random Forest model has yielded the best results, being the fastest model (both in inference and training), the lightest model, and the most adaptable. This model can be used in systems with limited resources, such as standalone VR glasses like Meta Quest or more powerful computers, and even in AI ecosystems due to its possible translation to TensorFlow already implemented.

In the analysis of the Random Forest results, it has been observed that the model places significant importance on the y-coordinate of the left leg and the y-coordinate of the left foot in the second time interval. This may indicate that gestures heavily depend on leg positions to distinguish between running, dancing, waving, or pointing.

Also, there is a slight confusion between the gestures of fighting and waving. This may be due to the rapid hand movements in both types of gestures, which, due to the lack of fingers in the suit, can lead to confusion between a punch and a wave.

4.6.5. Final Application Conclusions

The final application is a demo that connects to a server to display the results.

The negative aspect of this application is the way it connects to the server where the model is hosted, as it makes direct web requests to the Tensor Serving server, making it dependent on that server.

Finally, the final application fulfills its purpose of showing the user the predicted animation with minimal latency.

4.6.6. Limitations

This study has faced a series of limitations, including the scarcity of public datasets, imbalance in types of animations, lack of more data from real users, training issues with neural network models, and data shortages for those same models.

The limitation of dataset scarcity is not only due to the lack of datasets in number but also to the lack of gestures related to non-verbal communication. Most

available datasets focus on sports or very specific themes, with few general-purpose datasets. Another limitation of these datasets has been the ease of adaptation or conversion to a format compatible with the motion capture suit used in the study.

Regarding the imbalance of animation types, the issue arises from the duration of Mixamo's dance animations. Following the standardization process, the various dance animations are multiplied by a significantly larger number than those of other types. This was not resolved with the data collected from real users, as even though a large number of new animations were obtained, after standardization, dance animations still predominated significantly. This can be seen in Figures ?? and ??.

The neural network models did not yield the expected results, which may be due to the lack of data. Although there appears to be a sufficient number of samples at first glance, it is not enough for training neural networks. Additionally, hardware limitations for training more complex models and the time constraints of this study did not allow for longer training sessions or more extensive hyperparameter searches.

All these limitations have led to the following future work plan.

4.7. Future Work

The future work of this study can focus on several efforts:

- Improve the dataset by including people with functional diversity, increasing the number of samples per gesture, considering finger movements, and adding new categories. This could not only enhance the performance of neural network models but also provide greater differentiation for better generalization.
- Investigate other [AI](#) models that could improve classification accuracy and enable classification not only of gestures but also of emotions. This could help [NPCs](#) react more naturally to user gestures, making interactions more immersive.
- Standardize the server to be compatible with other [AI](#) technologies beyond TensorFlow. This would allow for the integration of emerging technologies into existing applications.
- Implement another model that considers not only the current predicted gesture but also the entire context to advance the use of non-verbal communication with [NPCs](#).
- Evaluation of the final result with users.

Contribuciones Personales

Alejandro Barrachina Argudo

Alejandro Barrachina, estudiante del Grado de Ingeniería Informática, se ha involucrado en el desarrollo del estudio desde el momento en que se le propuso la idea. Desde el principio, ha estado en comunicación constante con su compañero y sus tutores para comunicar los avances y dudas del proyecto. A lo largo del proyecto, estas son las tareas que ha desempeñado:

En un primer momento, se dedicó a estudiar el funcionamiento de Unity en gafas de VR y su posible integración con el traje de captura de movimiento y los modelos de IA pensados. Esto implicó aprender sobre Unity, C#, y servidores en Unity.

Una vez tuvo claro el comportamiento y funcionamiento de Unity, junto a su compañero, se dedicó a buscar posibles datasets para entrenar los modelos de IA. Tras varias búsquedas, cuando se optó por crear un dataset a mano, se dedicó a buscar herramientas para automatizar el proceso.

Al encontrar la herramienta Mixamo Downloader, la analizó hasta encontrar los errores de funcionamiento de la misma. Tras arreglarlos y añadir nuevas funcionalidades, se dedicó a crear la primera versión del dataset con los datos descargados, decidiendo la estructura de carpetas y su método de guardado.

Para este método de guardado, se decidió usar Kaggle para evitar problemas con Github y su límite de tamaño por archivo. Alejandro se ha encargado también de la estructura de repositorios y herramientas en la organización creada para este TFG, *FratosVR*, para así facilitar el desarrollo e interoperabilidad de las distintas herramientas con un desarrollo paralelo.

Tras investigar distintos estudios de IA para la detección de gestos, Alejandro decide hacer una comparativa entre los cuatro modelos expuestos en el estudio, reservándose otros por si hubiera más tiempo o recursos.

Mientras Pablo Sánchez desarrollaba la herramienta de conversión de datos, Alejandro investigó como hacer un *pipeline* de descarga del dataset, conversión al formato deseado usando Unity y su posterior actualización en el nuevo conjunto de datos de Kaggle. Para ello tuvo que investigar las funcionalidades de línea de comando de Unity, así como el funcionamiento del editor de Unity y sus funcionalidades expuestas en su API de C#.

Una vez creados estos datos, Alejandro discutió con sus tutores cual sería la

mejor manera de estandarizar los datos para su uso en los entrenamientos de los modelos. Cuando se llegó a un formato concreto, Alejandro se dedicó a programar la clase `DataLoader` para procesar todos los datos, estandarizarlos y actualizarlos en la nube para su posterior uso.

Tras esto, Alejandro modeló los cuatro modelos a probar y comparar. Esto implicó el uso de `Gradio` y `TensorBoard` para facilitar el entrenamiento para usuarios ajenos al proyecto. Esta idea se desarrolló por la ayuda cedida por la asociación LAG, que consistió en dejarnos equipos con gráficas más potentes que las que teníamos disponibles para así acelerar el entrenamiento de los modelos.

Mientras que Pablo se dedicaba a la adaptación de la herramienta para su uso con personas reales, Alejandro hizo dos formularios para las pruebas con usuarios. Uno de ellos para que las personas interesadas dejaran su correo y las fechas que tenían disponibles, y otro para la posterior recogida de datos demográficos. Estos formularios se pasaron a María del Carmen Fernández Villalba, Responsable de Protección de datos en Vicepresidencia Primera, perteneciente a Presidencia de La Junta de Comunidades de Castilla La Mancha, para su revisión y aprobación. Esto se hizo para asegurar que ninguno de los formularios incumplía la ley de protección de datos y así asegurar el anonimato de los datos recogidos y de los usuarios participantes. Tras sus sugerencias, se realizaron los cambios pertinentes y se abrieron para respuesta.

De manera adicional, empezó a plantear los diferentes modelos a entrenar y su representación gráfica en la web de entrenamiento. Para ello, investigó YDF, ya que era una herramienta nueva que no había usado, y desarrolló los modelos con TensorFlow de manera que fueran todos iguales de cara a la interfaz. Para ello, tuvo que investigar también `keras-tuner` para hacer que los entrenamientos fueran lo más eficientes posibles teniendo en cuenta las restricciones de hardware.

Una vez Pablo adaptó la herramienta para las pruebas con usuarios, Alejandro creo parte de los carteles para la difusión de las pruebas. Junto a Pablo hicieron una ruta por todas las facultades de la UCM de Ciudad Universitaria para colgarlos. También inició una campaña por sus redes sociales junto a las asociaciones de estudiantes de la facultad de informática para atraer a más gente.

Previo a las pruebas con usuarios, Alejandro junto a su compañero habló con los tutores y con distinto personal de la facultad para conseguir un espacio reservado el mayor tiempo posible para realizar dichas pruebas.

Durante las pruebas, tanto Alejandro como Pablo estuvieron presentes en todas para ayudar a los usuarios a ponerse el traje, realizar el calibrado del mismo y recoger las preguntas del cuestionario. Durante este proceso de pruebas, tanto Alejandro como Pablo se encargaron de atraer usuarios de prueba nuevos en caso de que fallaran los que tenían prueba asignada, resultando en que solo 5 plazas no fueran completadas. También se encargaron de gestionar horarios para maximizar el número de pruebas realizadas y gestionar la espera de los usuarios que se solaparan.

Tras esto, Alejandro se encargó de los entrenamientos de los modelos y los problemas que surgieron durante el proceso (Falta de VRAM, problemas de ingesta de datos, etc). También se encargó de desplegar los mecanismos de entrenamiento en distintos sistemas para no ocasionar inconvenientes a los usuarios de la asociación. Esto consistió en montar [WSL](#) en los ordenadores de la asociación para compati-

lizarlos con las últimas versiones de TensorFlow.

Una vez se acabaron los entrenamientos y se decidió el mejor modelo, Alejandro investigó una manera de hacer que estos modelos fueran multiplataforma y pudieran ser usados en cualquier dispositivo. Esto resultó en el descubrimiento e investigación de TensorFlow Serving. Alejandro posteriormente hizo scripts para Windows y Linux para facilitar el despliegado del modelo en docker usando las *releases* de Github.

Tras finalizar todo esto, Alejandro se dedicó a la redacción de este documento junto a su compañero, garantizando así cohesión durante todo el texto y ayuda con L^AT_EX en los momentos necesarios. También se dedicó a hacer las gráficas explicativas de los modelos y los datos demográficos para asegurar un entendimiento más sencillo de todos los resultados.

Pablo Sánchez Martín

Pablo Sánchez Martín, estudiante del Grado de Desarrollo de Videojuegos, se ha involucrado en el proyecto desde el momento en el que salió la idea manteniendo una comunicación constante con su compañero y tutores.

Desde el año 2023 mantuvo contacto con uno de los tutores, Alejandro Romero, para hacer un proyecto con un traje de captura de movimiento. Tras varias reuniones debatiendo lo que podría ser un trabajo interesante los dos decidieron el estudio actual. Para ello Pablo contactó al otro tutor del proyecto, Ismael Sagredo, e involucró a su compañero Alejandro Barrachina.

Durante el desarrollo estas son las tareas que ha realizado:

La primera tarea que tuvo que realizar junto a su compañero Alejandro era establecer qué gestos se querían detectar, qué modelos de IA se iban a utilizar y si se iba a detectar el movimiento de los dedos mediante las gafas de VR (handtracking). Finalmente los dos acordaron los cinco gestos en los que se basa el proyecto, los cuatro modelos con los que se hace la comparación y la decisión de no tener el handtracking por posibles problemas a la hora de no estar viendo las manos en todo momento.

Lo siguiente que hizo fue investigar el funcionamiento del traje de captura de movimiento junto al software necesario, tanto Axis Studio como el plugin de Unity. Para ello estuvo investigando las distintas funcionalidades dentro de un proyecto de Axis Studio que podría interesar para el estudio y se descargó un proyecto cedido por el tutor Alejandro Romero en el que se usaba el plugin para poder analizarlo y saber cómo usarlo.

Una vez entendió el funcionamiento del software necesario para el funcionamiento del traje empezó a buscar distintos datasets de animaciones para entrenar los modelos junto a su compañero. Al ver la escasez de estos datasets se optó por sacarlos de un banco de animaciones, por lo que empezó el desarrollo de la herramienta Mixamo Dumper.

En un primer momento investigó sobre el uso de los Assets Bundles de Unity para la carga automática de recursos externos a la aplicación, pero al final decidió no usar ese método. Finalmente para esa aplicación decidió usar el Asset Database de Unity, por lo que se dedicó a la investigación de esta API y finalmente a la

implementación de la herramienta. Una vez funcionaba la implementación de la carga de animaciones en tiempo de ejecución investigó como poder ejecutar todas estas animaciones seguidas en tiempo de ejecución, hasta que finalmente encontró el componente “Animator Override Controller”, lo que le permitió cumplir el objetivo de la herramienta.

Debido a los pocos datos encontrados en Mixamo se decidió grabar a usuarios con el traje de captura de movimiento, por lo que hizo en Unity una herramienta sencilla para crear [CSVs](#) en tiempo real con el traje de captura de movimiento.

Para las pruebas con usuarios se encargó de la comunicación con sus tutores y personal de la Facultad de Informática para conseguir un lugar en el que poder grabar. Con el fin de captar usuarios junto a su compañero hizo carteles y ayudó en la campaña en redes sociales y difusión por el resto de facultades de Ciudad Universitaria.

En la prueba explicó junto a su compañero a todos los usuarios el propósito de las pruebas, les ayudó a ponerse el traje y a explicarles el proceso de calibración. También le fue dando indicaciones a los usuarios sobre los gestos a realizar, grabó las animaciones y verificó que no hubiese ningún problema con estas. Adicionalmente se encargó junto a su compañero de buscar nuevos usuarios para cubrir huecos que habían quedado libres con el fin de maximizar el número de datos recogidos y gestionar a las personas que solapaban con otros usuarios.

Una vez recogidos los datos investigó diferentes formas de conectarse con el servidor que hostee el modelo elegido. Investigó el uso de “Unity NetCode”, un paquete propio de Unity para la comunicación en red en videojuegos y también investigó la posibilidad de realizar la comunicación mediante las llamadas nativas de C#. Finalmente se decidió usar la arquitectura [API REST](#), por lo que investigó su funcionamiento mediante el paquete nativo “UnityWebRequest” y empezó la aplicación final con la implementación de ésta en Unity.

Una vez estaba hecha la comunicación con el servidor diseñó e implementó una aplicación final con soporte para las gafas de [VR](#) y el traje de captura de movimiento en el que se conectase con el servidor y se pueda ver el resultado del modelo elegido en tiempo real y como un [NPC](#) reacciona a los gestos predichos por éste.

Finalmente ha contribuido a la redacción de este documento junto a su compañero, así como la constante comunicación con los tutores para comprobar que el formato de éste era adecuado y a la realización de distintos diagramas para que las herramientas creadas durante el proceso del proyecto sean más sencillas de entender.

Bibliografía

The sciences, each straining in its own direction, have hitherto harmed us little; but some day the piecing together of dissociated knowledge will open up such terrifying vistas of reality, and of our frightful position therein, that we shall either go mad from the revelation or flee from the deadly light into the peace and safety of a new dark age.

H.P. Lovecraft, *The Call of Cthulhu*

ABADI, M., AGARWAL, A., BARHAM, P., BREVDO, E., CHEN, Z., CITRO, C., CORRADO, G. S., DAVIS, A., DEAN, J., DEVIN, M., GHEMAWAT, S., GOODFELLOW, I., HARP, A., IRVING, G., ISARD, M., JIA, Y., JOZEFOWICZ, R., KAISER, L., KUDLUR, M., LEVENBERG, J., MANÉ, D., MONGA, R., MOORE, S., MURRAY, D., OLAH, C., SCHUSTER, M., SHLENS, J., STEINER, B., SUTSKEVER, I., TALWAR, K., TUCKER, P., VANHOUCHE, V., VASUDEVAN, V., VIÉGAS, F., VINYALS, O., WARDEN, P., WATTENBERG, M., WICKE, M., YU, Y. y ZHENG, X. TensorFlow: Large-scale machine learning on heterogeneous systems. 2015. Software available from [tensorflow.org](https://www.tensorflow.org).

ANSEL, J., YANG, E., HE, H., GIMELSHEIN, N., JAIN, A., VOZNESENSKY, M., BAO, B., BELL, P., BERARD, D., BUROVSKI, E., CHAUHAN, G., CHOURDIA, A., CONSTABLE, W., DESMAISON, A., DEVITO, Z., ELLISON, E., FENG, W., GONG, J., GSCHWIND, M., HIRSH, B., HUANG, S., KALAMBARKAR, K., KIRSCH, L., LAZOS, M., LEZCANO, M., LIANG, Y., LIANG, J., LU, Y., LUK, C., MAHER, B., PAN, Y., PUHRSCHE, C., RESO, M., SAROUFIM, M., SIRAI-CHI, M. Y., SUK, H., SUO, M., TILLET, P., WANG, E., WANG, X., WEN, W., ZHANG, S., ZHAO, X., ZHOU, K., ZOU, R., MATHEWS, A., CHANAN, G., WU, P. y CHINTALA, S. PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation. En *29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '24)*. ACM, 2024.

ELLIS, S. R. Nature and origins of virtual environments: a bibliographical essay. *Computing Systems in Engineering*, vol. 2, páginas 321–347, 1991.

- GUERRA-FILHO, G. Optical motion capture: Theory and implementation. *Rita*, vol. 12(2), páginas 61–90, 2005.
- GUILLAME-BERT, M., BRUCH, S., STOTZ, R. y PFEIFER, J. Yggdrasil decision forests: A fast and extensible decision forests library. En *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, KDD 2023, Long Beach, CA, USA, August 6-10, 2023*, páginas 4068–4077. 2023.
- HARRIS, C. R., MILLMAN, K. J., VAN DER WALT, S. J., GOMMERS, R., VIRTANEN, P., COURNAPEAU, D., WIESER, E., TAYLOR, J., BERG, S., SMITH, N. J., KERN, R., PICUS, M., HOYER, S., VAN KERKWIJK, M. H., BRETT, M., HALDANE, A., DEL RÍO, J. F., WIEBE, M., PETERSON, P., GÉRARD-MARCHANT, P., SHEPPARD, K., REDDY, T., WECKESSER, W., ABBASI, H., GOHLKE, C. y OLIPHANT, T. E. Array programming with NumPy. *Nature*, vol. 585(7825), páginas 357–362, 2020.
- HERBERT, O. M., PÉREZ-GRANADOS, D., RUIZ, M. A. O., CADENA MARTÍNEZ, R., GUTIÉRREZ, C. A. G. y ANTUÑANO, M. A. Z. Static and dynamic hand gestures: A review of techniques of virtual reality manipulation. *Sensors*, vol. 24(12), página 3760, 2024.
- HOLFELD, J. On the relevance of the godot engine in the indie game development industry. <https://arxiv.org/abs/2401.01909>, 2024.
- JOCHER, G., QIU, J. y CHAURASIA, A. Ultralytics YOLO. 2023.
- MA, X., ZENG, B. y XING, Y. Combining 3d skeleton data and deep convolutional neural network for balance assessment during walking. *Frontiers in Bioengineering and Biotechnology*, vol. 11, 2023.
- WES MCKINNEY. Data Structures for Statistical Computing in Python. En *Proceedings of the 9th Python in Science Conference* (editado por Stéfan van der Walt y Jarrod Millman), páginas 56 – 61. 2010.
- NEVEROVA, N., WOLF, C., PACI, G., SOMMAVILLA, G., TAYLOR, G. W. y NEBOUT, F. A multi-scale approach to gesture detection and recognition. 2013.
- OLSTON, C., FIEDEL, N., GOROVY, K., HARMSSEN, J., LAO, L., LI, F., RAJASHEKHAR, V., RAMESH, S. y SOYKE, J. Tensorflow-serving: Flexible, high-performance ml serving. <https://arxiv.org/abs/1712.06139>, 2017.
- PALLAVICINI, F., PEPE, A. y MINISSI, M. E. Gaming in virtual reality: What changes in terms of usability, emotional response and sense of presence compared to non-immersive video games? *SIMULATION AND GAMING*, vol. 50, páginas 136–159, 2019.
- PEDREGOSA, F., VAROQUAUX, G., GRAMFORT, A., MICHEL, V., THIRION, B., GRISEL, O., BLONDEL, M., PRETTENHOFER, P., WEISS, R., DUBOURG, V., VANDERPLAS, J., PASSOS, A., COURNAPEAU, D., BRUCHER, M., PERROT, M. y DUCHESNAY, E. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, vol. 12, páginas 2825–2830, 2011.

- RAAB, F. H., BLOOD, E. B., STEINER, T. O. y JONES, H. R. Magnetic position and orientation tracking system. *IEEE Transactions on Aerospace and Electronic Systems*, vol. AES-15(5), páginas 709–718, 1979.
- ROETENBERG, D., LUINGE, H., SLYCKE, P. ET AL. Xsens mvn: Full 6dof human motion tracking using miniature inertial sensors. *Xsens Motion Technologies BV, Tech. Rep.*, vol. 1(2009), páginas 1–7, 2009.
- PANDAS DEVELOPMENT TEAM, T. pandas-dev/pandas: Pandas. 2020.
- VAN BRAKEL, V., BARREDA-ÁNGELES, M. y HARTMANN, T. Feelings of presence and perceived social support in social virtual reality platforms. *Computers in Human Behavior*, vol. 139, 2023.
- VRCHAT, I. Vr chat full-body tracking. <https://docs.vrchat.com/docs/full-body-tracking>, 2024. Accessed: (03/05/2025).
- WILKINSON, M., BRANTLEY, S. y FENG, J. A mini review of presence and immersion in virtual reality. *Human Factors and Ergonomics Society*, 2021.
- XENAKIS, I., GAVALAS, D., KASAPAKIS, V., DZARDANOVA, E. y VOSINAKIS, S. Non-verbal communication in immersive virtual reality through the lens of presence: A critical review. *PRESENCE: Virtual and Augmented Reality*, vol. 31, páginas 1–71, 2023.
- ZHANG, Z. Microsoft kinect sensor and its effect. *IEEE MultiMedia*, vol. 19(2), páginas 4–10, 2012.

JSON Schema para los Behavior Trees generados

Listing A.1: JSON Schema (Draft 2020-12) de los Behavior Trees generados

```

1  {
2    "$schema": "https://json-schema.org/draft/2020-12/schema",
3    "$id": "https://example.com/behaviour-tree.schema.json",
4    "title": "Behaviour Tree",
5    "type": "object",
6    "required": ["Blackboard", "Node"],
7    "additionalProperties": false,
8
9    "properties": {
10     "Blackboard": {
11       "type": "array",
12       "description": "Blackboard",
13       "items": {
14         "type": "object",
15         "additionalProperties": {
16           "type": "string",
17           "enum": ["Bool", "Int", "Float", "Vector",
18             ↪ "Object", "String"]
19         }
20       }
21     },
22     "Node": {
23       "$ref": "#/$defs/TreeNodeWrapper"
24     }
25   },
26
27   "$defs": {
28     "TreeNodeWrapper": {
29       "type": "object",
30       "description": "Nodo del arbol",

```

```

31     "minProperties": 1,
32     "maxProperties": 1,
33     "additionalProperties": {
34         "$ref": "#/$defs/TreeNode"
35     }
36 },
37
38 "TreeNode": {
39     "type": "object",
40     "required": ["Type", "Decorators", "Nodes"],
41     "additionalProperties": false,
42
43     "properties": {
44         "Type": {
45             "type": "string",
46             "enum": ["Selector", "Sequence", "Task", "Parallel"]
47         },
48
49         "Name": {
50             "type": "string",
51             "description": "Nombre opcional del nodo"
52         },
53
54         "Decorators": {
55             "type": "array",
56             "items": {
57                 "type": "object",
58                 "additionalProperties": {
59                     "type": "string"
60                 }
61             }
62         },
63
64         "Task": {
65             "type": "string",
66             "description": "Nombre de la tarea"
67         },
68
69         "BlackboardEntries": {
70             "type": "array",
71             "description": "Entradas del Blackboard usadas por  

72                 ↪ el nodo",
73             "items": {
74                 "type": "string"
75             }
76         },
77
78         "Nodes": {
79             "type": "array",

```

```
79         "description": "Nodos hijos",
80         "items": {
81             "$ref": "#/$defs/TreeNodeWrapper"
82         }
83     }
84 }
85 ,
86 "Nodes": {
87     "type": "array",
88     "description": "Nodos hijos",
89     "minItems": 0,
90     "items": {
91         "type" : "object",
92         "required": ["Node"],
93         "additionalProperties" : false,
94         "properties": {
95             "Node": {
96                 "$ref": "#/$defs/TreeNode"
97             }
98         }
99     }
100 },
101
102 "allOf": [
103     {
104         "if": {
105             "properties": { "Type": { "const": "Task" } }
106         },
107         "then": {
108             "required": ["Task"]
109         }
110     }
111 ]
112 }
113 }
114 }
```


Glosario

Abstract Factory	El patrón Abstract Factory es un patrón de programación que consiste en mantener una clase que permita crear instancias de objetos relacionados gracias al polimorfismo.
AI	Artificial Intelligence
API	Application Programming Interface
API REST	Arquitectura de diseño utilizada para crear servicios web y comunicar diferentes sistemas
Behavior Tree	Un Behavior Tree es un modelo de ejecución de tareas de forma jerárquica basado en árboles usada en el campo de la robótica y la inteligencia artificial
Blackboard	La Blackboard es la memoria compartida usada en un Behavior Tree. Ésta funciona como un diccionario de pares clave-valor, que pueden ser variables de entrada/salida.
CART	Un árbol de decisión CART (Classification and Regression Trees) es un algoritmo de aprendizaje automático que utiliza árboles binarios para clasificar o predecir valores continuos
CNN	Convolutional Neural Network
CPU	Central Processing Unit
CSV	Comma Separated Values
DCNN	Deep Convolutional Neural Network
GPU	Graphics Processing Unit
gradient boosted decision tree	Un gradient boosted decision tree es un algoritmo de aprendizaje automático que utiliza múltiples árboles de decisión y optimización por gradiente para mejorar la precisión de las predicciones
IA	Inteligencia Artificial

IRs	Rayos Infrarrojos
isolation forest	Un isolation forest es un algoritmo de aprendizaje automático que utiliza árboles de decisión para detectar anomalías en los datos
LLM	Un LLM (Large Language Model) es un modelo de aprendizaje automático profundo (deep learning) que ha sido entrenado con grandes cantidades de texto con propósito general de predicción de palabras.
LSTM	Long Short-Term Memory
metaverse	The metaverse is a set of digital spaces to socialize, learn, play and do other activities. (Definition by the company Meta)
metaverso	El metaverso es un conjunto de espacios digitales para socializar, aprender, jugar y realizar otras actividades. (Definición de la empresa Meta)
MMORPG	Massively Multiplayer Online Role-Playing Game
NPC	Non Playable Character
Random Forest	Un random forest es un algoritmo de aprendizaje automático que utiliza múltiples árboles de decisión para mejorar la precisión y la robustez de las predicciones
SDK	Software Development Kit
Singleton	El patrón Singleton es un patrón de programación que consiste en garantizar que una clase tenga una instancia única y que se pueda acceder globalmente a ésta.
SVM	Support Vector Machine
TPU	Tensor Processing Unit
VR	Realidad Virtual
WSL	Windows Subsystem for Linux
YOLO	You Only Look Once

Licencia de uso del documento

Esta obra está bajo una licencia [Creative Commons](http://creativecommons.org/licenses/by-sa/4.0/legalcode) «Atribución-NoComercial-CompartirIgual 4.0 Internacional».



<http://creativecommons.org/licenses/by-sa/4.0/legalcode>.

Licencia de uso del código fuente

Los archivos de código fuente para generar este documento se encuentran en <https://github.com/FratosVR/Memoria-TFG> bajo la licencia [GPL-3.0](#)